arXiv:2607.11498v2 [cs.RO] 21 Sep 2026

See like a Robot: Robot-Centric Pointmaps for VLA Models

Byungkun Lee[1,*,†], Dongyoon Hwang[1,*], Dongjin Kim[1], Hojoon Lee[2], Hyunseung Kim[1,3], Jaegul Choo[1], and Minho Park[1]

[1]KAIST AI [2]Holiday Robotics [3]KRAFTON AI

***Abstract*— Vision-language-action (VLA) models require 3D spatial reasoning, yet RGB observations encode robot-object geometry only implicitly. Lifting depth with camera intrinsics makes this geometry explicit as dense, image-aligned pointmaps, but their camera-frame coordinates depend on camera placement. We propose SeeR-VLA, which transforms pointmaps into a robot-centric frame with an end-effector origin and robot-base-aligned axes. An encoder initialized from pretrained RGB weights extracts pointmap features, which are added to corresponding RGB tokens without increasing the token count. Across 24 RoboCasa tasks and four real-world tasks, SeeR-VLA improves average success over RGB-only $\pi_{0.5}$ by 6.4 and 32.5 percentage points, respectively. It also exceeds the strongest evaluated 3D-augmented baseline, PointVLA, by 3.5 and 23.7 percentage points, respectively. Beyond these gains, our ablations clarify how coordinate choices affect VLA performance, showing that end-effector centering is most effective with robot-base-aligned axes. The benefits grow as training viewpoints diversify, highlighting the importance of using robot-frame pointmaps when learning from diverse camera configurations. The project page is available here.**

I. Introduction

Vision-language-action (VLA) models learn robotic manipulation policies from large-scale datasets, predicting actions from visual observations and language instructions [1]–[3]. Accurate action prediction requires reasoning about metric 3D geometry, such as the target object's position relative to the gripper during grasping. However, as shown in Fig. 1, RGB observations encode this geometry only implicitly, requiring policies to infer object poses and their spatial relationships to the gripper from robot data with limited direct geometric supervision. This gap has motivated growing interest in equipping VLA models with explicit 3D information [4]–[10].

Depth is widely used to provide explicit 3D information for manipulation [4]–[6], [10]–[12], but a scalar depth value alone does not specify a pixel's 3D position. Recovering this position requires back-projecting the pixel using its depth and camera intrinsics. The resulting 3D coordinates are commonly processed as point clouds, which explicitly represent scene geometry but are often downsampled into sparse point sets to reduce computational cost. This sampling can discard fine geometric details needed for precise manipulation. Pointmaps instead preserve the dense output of depth back-projection by storing each pixel's 3D coordinates in an image-aligned $H \times W \times 3$ grid. Constructed directly from depth and camera intrinsics, this representation retains dense geometry and allows standard image encoders to combine it with visual features at corresponding locations. These properties have motivated increasing use of pointmaps in 3D vision [13]–[18] and robot policy learning [19]–[21].

However, pointmaps express 3D positions in the camera frame, so the same scene point has different coordinates when observed from different camera placements. This variation is particularly relevant to VLAs pretrained on large-scale robot datasets collected with diverse camera configurations [22]–[24]. A common solution is to express pointmaps in the robot base frame, making their coordinates independent of camera placement [19]. For manipulation, however, the target's position relative to the end effector is particularly important [25]. Centering pointmaps at the end effector directly captures this relationship by describing grasp targets and articulated parts relative to a common interaction origin [26]. This representation preserves the coordinates of the same interaction geometry across changes in camera placement or interaction location within the workspace, providing a consistent geometric reference for learning manipulation.

To this end, we propose SeeR-VLA, which augments pretrained VLAs with robot-centric pointmaps that combine explicit interaction geometry with image-aligned visual features. We construct pointmaps from depth and camera calibration, placing the origin at the end effector while keeping the coordinate axes aligned with the robot base. Each point thus describes a displacement from the gripper without changing the axis directions as the gripper rotates. To incorporate this geometry into the VLA, we process pointmaps with a separate encoder initialized from the pretrained RGB encoder. The shared image grid allows us to add pointmap features to RGB features at corresponding spatial locations, pairing visual appearance with the position of the observed surface relative to the end effector. These fused features replace the original RGB tokens, providing explicit 3D information without increasing the visual token count or changing the policy's action prediction objective.

We evaluate SeeR-VLA on 24 RoboCasa tasks and four real-world tasks collected with diverse training camera viewpoints. SeeR-VLA outperforms PointVLA [7], the strongest evaluated 3D-augmented VLA baseline, by 3.5 percentage points on RoboCasa and 23.7 percentage points on real-world tasks. Controlled ablations also show that explicitly constructing pointmaps is more effective than supplying depth and camera cues for the policy to learn the geometric conversion, and that retaining the image grid outperforms

*These authors contributed equally.
†Corresponding author.

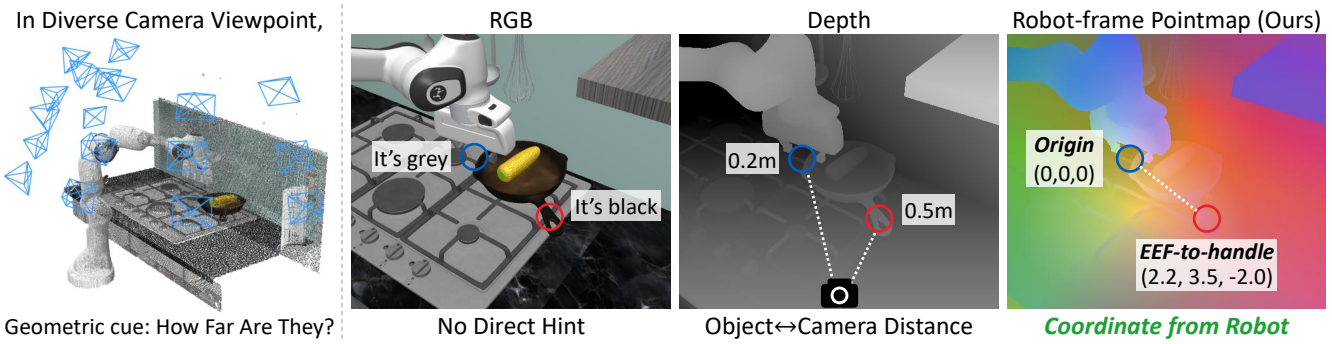


Fig. 1. **Depth versus robot-centric pointmaps.** Depth measures camera-relative distance, whereas our pointmap stores pixel-aligned 3D coordinates relative to the end effector with robot-base-aligned axes, making the represented interaction geometry invariant to training-time camera viewpoint.

point-cloud encoding. The best results are obtained by combining an end-effector origin, robot-base-aligned axes, and additive RGB-pointmap fusion. Importantly, the advantage over RGB-only policies grows as training camera viewpoints become more diverse, supporting the value of a consistent geometric representation. Masking and action-sensitivity analyses further reveal that the policy selectively relies on target-object and robot coordinates rather than irrelevant background geometry.

We make the following contributions.

- We introduce SeeR-VLA, which augments VLAs with robot-centric pointmaps through RGB-pointmap fusion for manipulation across diverse camera viewpoints.
- We identify an effective pointmap design through controlled ablations, combining an end-effector origin, robot-base-aligned axes, and additive RGB-pointmap fusion on the image grid.
- We show that robot-centric pointmaps improve robustness to training-time viewpoint variation, with increasing gains over RGB-only policies as viewpoints diversify.

II. Related Work

1) 3D Geometric Observation for Robotics: Robotic manipulation policies use diverse 3D representations to capture geometry beyond RGB appearance. Depth provides dense, image-aligned cues [4], [5], [10], but recovering explicit 3D coordinates requires back-projection using camera intrinsics. Point-cloud methods process the resulting geometry [9], [11], [12], [27]–[29], often subsampling it for efficiency and potentially discarding fine geometric details. Other approaches use voxel grids [30], 3D feature fields [31], virtual views [6], or unified multi-view representations [32]. Pointmaps instead organize back-projected 3D coordinates in a dense $H \times W \times 3$ image grid, preserving RGB-pixel correspondence and allowing standard image encoders to process the geometry. These properties have motivated their use in 3D vision [13]–[18], [33], [34] and robot policy learning [19]–[21].

Coordinate frames also determine how this geometry is expressed to the policy. PointMapPolicy [19] represents dense pointmaps in a shared world frame, while FP3 [35] uses robot-base-frame point clouds, aligning metric coordinates across camera views. However, geometrically similar interactions can still be represented by different coordinates when they occur at different workspace locations. Frame Mining [26] systematically studies coordinate-frame choices for pointcloud-based manipulation and shows that frames beyond the robot-base frame, such as end-effector and target-centered frames, can improve policy learning. Related work uses object-centric trajectories [36], affordance-relative states and actions [37], or canonicalized point-cloud inputs [38]. Adapt3R [39] transforms semantic point clouds into the end-effector coordinate frame. We study coordinate-frame design for dense pointmaps in pretrained VLAs, separately varying the origin and axis orientation to distinguish end-effector centering from rotation of the axes with the gripper.

2) Injecting 3D Geometry into VLAs: Beyond selecting a geometric representation and reference frame, incorporating geometry into pretrained VLAs requires deciding how it interacts with existing features. Camera-aware methods provide calibration-derived cues [40], [41] or express actions in the camera frame [42], whereas scene-geometry-aware methods enrich visual representations with depth-derived spatial embeddings [4], [43]. Other approaches introduce dedicated geometric encoders or experts [5], [7]–[10], [29], [44], render geometry into image inputs [45], [46], or incorporate future 3D scene generation [47]. Geometric foundation models offer another route through feature injection [48] or training-time representation alignment [49]. Despite these advances, the integration of image-aligned pointmaps into pretrained VLAs remains underexplored. We investigate this integration by initializing the pointmap encoder with pretrained RGB-encoder weights and comparing fusion strategies, without requiring point-cloud-specific architectures or additional rendering or scene-generation stages.

III. Method

To bridge the coordinate-frame mismatch between camera-centric observations and robot-centric actions, SeeR-VLA converts multi-view RGB-D observations into explicit 3D pointmaps expressed in the robot coordinate frame, as illustrated in Fig. 2. We first introduce the problem setup and notation used throughout the method (Section III-A). We then construct a pointmap for each view from its depth observation and camera intrinsics (Section III-B), and transform the

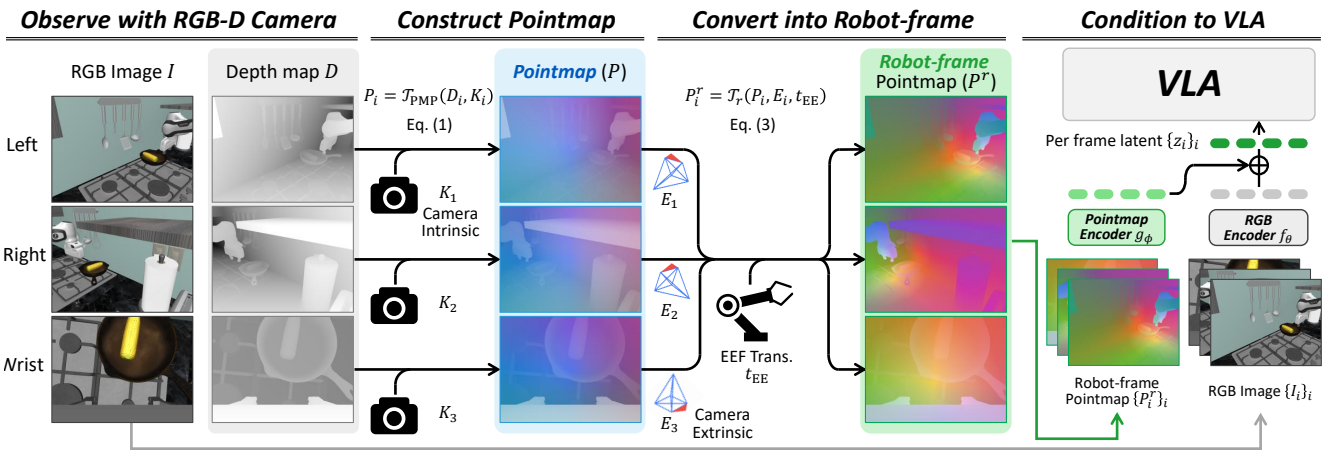


Fig. 2. **Overview of the SeeR-VLA framework.** We lift depth into camera-frame pointmaps using intrinsics, then construct robot-frame pointmaps by applying extrinsics and shifting the origin to the end effector while retaining robot-base-aligned axes. Next, the RGB encoder f_θ and pointmap encoder g_ϕ extract corresponding features. Finally, we add these features to form z_i, replacing the VLA's RGB tokens without increasing the token count.

resulting pointmaps into a common robot coordinate frame (Section III-C). Finally, we condition the VLA on these robot-centric pointmaps to provide explicit 3D geometric information aligned with the coordinate frame in which robot actions are executed (Section III-D).

A. Problem Setup

We consider a multi-view robotic manipulation setting with N RGB-D cameras. At each timestep, the robot observes RGB images $\{I_i\}_{i=1}^N$ and their corresponding depth maps $\{D_i\}_{i=1}^N$. For each camera i, we assume known intrinsics K_i and extrinsics $E_i := T^{r \leftarrow c_i}$, where $T^{r \leftarrow c_i}$ denotes the transformation from the i-th camera coordinate frame to the robot coordinate frame. The robot proprioceptive state is denoted by S, from which the end-effector pose $(R_{\text{EE}}, t_{\text{EE}})$ is obtained via forward kinematics. We build on the VLA policy $\pi_{0.5}$ [3], augmenting its visual input with robot-frame pointmaps. The language instruction, robot proprioceptive state, and flow-matching training objective [50] otherwise follow the original VLA setup [3].

B. Pointmap Construction

Given the depth map D_i and camera intrinsics K_i of the i-th view, we construct a camera-frame pointmap P_i as

$$P_i = \mathcal{T}_{\text{PMP}}(D_i, K_i) \in \mathbb{R}^{H \times W \times 3}, \tag{1}$$

where $\mathcal{T}_{\text{PMP}}$ denotes the pixel-to-3D lifting operation. Specifically, for each pixel (u, v),

$$P_i(u, v) = D_i(u, v) K_i^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}. \tag{2}$$

Each element of P_i therefore stores the 3D coordinate of the corresponding pixel in the i-th camera coordinate frame. The resulting pointmap preserves the $H \times W$ spatial grid of the RGB image, providing a pixel-aligned geometric representation.

C. Robot-Centric Pointmap

The pointmap P_i is expressed in the local coordinate frame of the i-th camera. To obtain a view-consistent geometric representation aligned with the robot, we transform each pointmap into a robot-centric coordinate frame. Using the camera-to-robot extrinsic E_i and the current end-effector position t_{EE}, we define the robot-centric pointmap as

$$P_i^r = \mathcal{T}_r(P_i, E_i, t_{\text{EE}}), \tag{3}$$

where $\mathcal{T}_r$ first transforms the camera-frame coordinates into the robot base frame and then shifts the origin to the current end effector.

Let R_i and t_i denote the rotation and translation components of E_i, respectively. For each point in the pointmap, the transformation is given by

$$P_i^r(u, v) = R_i P_i(u, v) + t_i - t_{\text{EE}}. \tag{4}$$

Since all pointmaps are expressed in the same robot-centric coordinate system, a physical scene point has the same coordinate across training-time camera viewpoints. Moreover, centering the representation at the current end effector expresses each scene point by its displacement from the gripper rather than by its absolute location in the workspace. Consequently, interactions requiring similar end-effector motions induce similar local geometry even when they occur at different workspace locations. We ablate the effect of this robot-centric representation in Section IV-D.

D. Pointmap-RGB Fusion

Since the pointmap preserves the $H \times W$ spatial grid of the RGB image, it can be processed using the same encoder architecture as the RGB observation. We introduce a separate pointmap encoder g_ϕ that shares the architecture of the VLA's RGB encoder f_θ and is initialized with its pretrained weights. The two encoders produce tokens of the same shape, allowing us to fuse the geometric and visual features through element-wise addition:

$$z_i = f_\theta(I_i) + g_\phi(P_i^r) \in \mathbb{R}^{N_{\text{tok}} \times d}, \tag{5}$$

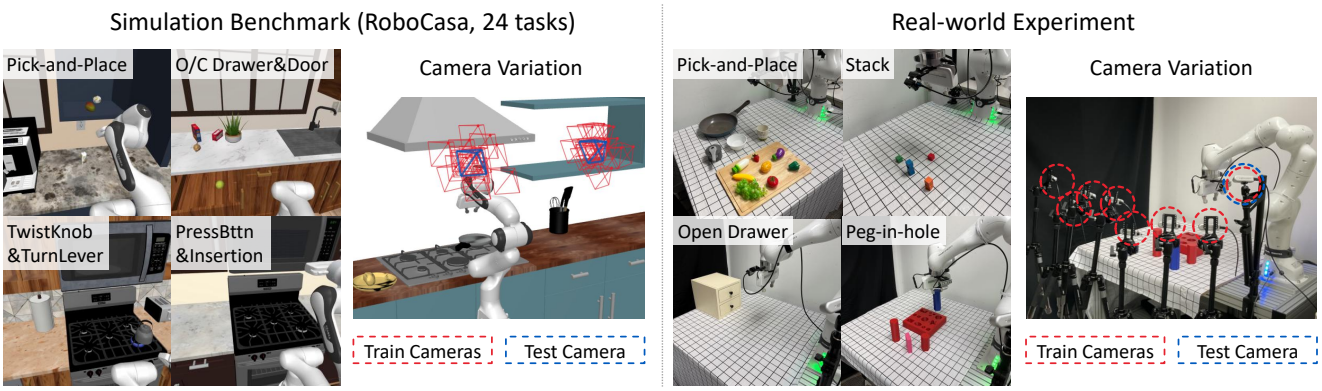


Fig. 3. **Simulation and real-world experimental setups.** Left, we evaluate on 24 RoboCasa tasks with camera placements randomized across training demonstrations (red) and fixed to a reference viewpoint during evaluation (blue). Right, our real-world tasks comprise pick-and-place, block stacking, drawer opening, and peg-in-hole insertion. Demonstrations are collected from multiple camera placements (red), and evaluation uses the rightmost placement (blue).

where N_{tok} and d denote the number and dimension of the visual tokens, respectively. The fused tokens z_i replace the original RGB tokens as the visual input to the VLA. Because the fusion is performed through element-wise addition, the number of visual tokens remains unchanged, allowing the pretrained VLA backbone to incorporate robot-centric 3D geometry without increasing its input sequence length.

IV. Experiments

Our experiments address the following questions.

- *RQ1:* How much do robot-centric pointmaps improve manipulation performance over RGB-only policies and existing 3D-augmented VLAs? (Section IV-B)
- *RQ2:* How do different 3D representations, pointmap-RGB fusion methods, and training-time viewpoint settings affect VLA performance? (Section IV-C)
- *RQ3:* How should the pointmap coordinate frame be defined to best support VLAs? (Section IV-D)
- *RQ4:* Which pointmap regions does SeeR-VLA rely on for action prediction? (Section IV-E)

A. Experimental Setup

1) Benchmarks: We evaluate SeeR-VLA in simulation and the real world with diverse training-time camera viewpoints. In simulation, we use 24 RoboCasa tasks [51] with 50 human demonstrations per task. Based on the RoboCasa atomic-skill taxonomy, we group them into four categories: pick-and-place (8 tasks), drawer/door opening and closing (6), knob/lever turning (5), and button pressing/insertion (5). The external cameras are randomized per demonstration and fixed to reference viewpoints at evaluation, following RoboCasa's default setup (Fig. 3, left). We report mean and standard deviation over three seeds, with 50 episodes per task per seed. For real-world experiments, we use a Franka Research 3 with a wrist camera (RealSense D405) and an external camera (RealSense D435i). We collect 320 demonstrations across four tasks and eight external camera viewpoints, with 10 demonstrations per task per viewpoint (Fig. 3, right). We evaluate each method over 20 trials per task at one training viewpoint and report success rates.

2) Baselines: We compare SeeR-VLA against RGB-only $\pi_{0.5}$ [3] and three groups of baselines incorporating viewpoint or 3D information. *(i) Camera-geometry-aware VLAs* include KYC [41], which provides per-pixel Plücker rays from camera intrinsics and extrinsics, and OC-VLA [42], which predicts actions in the camera frame. *(ii) Scene-geometry-augmented VLAs*, including GeoVLA [8] and PointVLA [7], process point clouds with dedicated 3D modules. *(iii) Point cloud policies* are represented by FP3 [35], which uses robot-frame point clouds without a pretrained VLM backbone. As official implementations are unavailable for groups *(i)* and *(ii)*, we reimplement them on the $\pi_{0.5}$ backbone for controlled comparison. For real-world experiments, we select OC-VLA and PointVLA as the strongest variants in their respective groups in simulation, alongside RGB-only $\pi_{0.5}$ and FP3.

3) Implementation Details: For the main experiments, we initialize $\pi_{0.5}$-based methods from pretrained $\pi_{0.5}$ (base) and FP3 from its DROID-pretrained checkpoint [24]. We fine-tune them for 20k steps on RoboCasa and 50k steps on real-world data with a batch size of 64. The pointmap encoder is initialized from the pretrained RGB encoder [52], and both are updated during training. The $\pi_{0.5}$-based methods use AdamW [53] ($\beta = (0.9, 0.95)$, weight decay 0.01) with cosine decay and learning rates of 10^{-4} in simulation and 2.5×10^{-5} in the real world. To isolate design choices from robot pretraining, ablations in Section IV-C and Section IV-D use a $\pi_{0.5}$-style architecture [3] initialized from base PaliGemma [54], with all variants trained for 30k steps.

Following OpenPI [3], $\pi_{0.5}$-based methods condition on o_t, predict 50 actions, and execute 25 before replanning. Following its official setup, FP3 conditions on (o_{t-1}, o_t) with 4000 points per camera, predicts 16 actions, and executes 8. For all real-world depth-based inputs, we use FastFoundationStereo [55] to reduce sensor noise. We discard depth beyond 10 m in simulation and 3 m for real-world external cameras; missing or discarded pixels are represented by a learned three-dimensional mask token in the pointmap.

TABLE I. COMPARISON WITH 3D-AUGMENTED VLAS ON 24 ROBOCASA AND FOUR REAL-WORLD TASKS.

	RoboCasa [51]					Real-world				
Method	Pick&Place	Open/Close	Knob/Lever	Button/Insert	Avg.	Pick & place	Stack	Open dr.	Peg in hole	Avg.
FP3 [35]	24.3 ±2.3	76.1 ±1.3	33.2 ±1.8	42.4 ±3.1	42.9 ±0.7	25.0	15.0	80.0	55.0	43.8
$\pi_{0.5}$ [3]	45.2 ±0.8	81.6 ±2.1	41.6 ±2.4	52.7 ±1.4	55.1 ±0.3	50.0	15.0	65.0	20.0	37.5
+ KYC [41]	47.0 ±5.2	83.1 ±3.2	43.5 ±5.1	49.6 ±5.7	55.8 ±3.9	—	—	—	—	—
+ OC-VLA [42]	52.8 ±2.4	80.8 ±0.8	**46.4** ±2.8	48.0 ±2.9	57.4 ±1.1	60.0	25.0	75.0	20.0	45.0
+ GeoVLA [8]	50.4 ±0.8	81.2 ±1.0	42.1 ±4.8	55.1 ±2.2	57.4 ±1.4	—	—	—	—	—
+ PointVLA [7]	47.7 ±1.8	86.6 ±0.2	42.5 ±2.8	55.7 ±1.0	58.0 ±0.7	45.0	30.0	75.0	35.0	46.3
+ SeeR-VLA (Ours)	**52.9** ±0.3	**88.0** ±1.7	46.1 ±1.4	**58.8** ±2.9	**61.5** ±1.2	**75.0**	**55.0**	**85.0**	**65.0**	**70.0**

B. Comparison with 3D-augmented VLAs

1) Simulation: Among methods sharing the $\pi_{0.5}$ backbone, the strongest camera-aware variant (OC-VLA [42], 57.4) and 3D-augmented variant (PointVLA [7], 58.0) both remain below SeeR-VLA (61.5). Notably, both PointVLA and FP3 [35] process geometry as point clouds rather than dense, RGB-aligned pointmaps. Their lower performance suggests that preserving dense spatial correspondence between RGB and geometry may be beneficial for manipulation. These methods differ in both geometric inputs and integration mechanisms, so Section IV-C isolates these design choices through controlled comparisons. FP3, the only baseline not built on $\pi_{0.5}$, further lacks large-scale vision-language pretraining and uses a sparse point cloud input (4000 points per view), compared with our 128×128 pointmap (16,384 points per view).

2) Real-world: Tab. I shows that adding robot-centric pointmaps raises the average success rate of $\pi_{0.5}$ from 37.5 to 70.0, achieving the highest success rate on all four tasks. PointVLA [7], the strongest baseline, reaches only 46.3 on average. The benefit of explicit 3D geometry is particularly evident on peg-in-hole, where PointVLA, FP3 [35], and SeeR-VLA achieve 35.0%, 55.0%, and 65.0%, respectively, compared with 20.0% for both RGB-only $\pi_{0.5}$ and OC-VLA [42]. While FP3 performs strongly on drawer opening (80.0%) and peg-in-hole (55.0%), it drops to 25.0% on pick-and-place and 15.0% on stacking. Our 640×480 pointmaps retain 307,200 points per view, over $75\times$ more than the 4000-point clouds used by PointVLA and FP3, while preserving pixel-wise correspondence with RGB. We further observe that FP3 failures often involve semantic object selection, such as choosing the wrong object or target. These results suggest that combining explicit geometry for precise manipulation with pretrained vision-language features for semantic reasoning is important, with SeeR-VLA consistently improving performance across all four tasks.

C. Ablation Studies

To examine the factors underlying these gains, we conduct controlled ablations on 3D representations, pointmap–RGB fusion, and training-time viewpoint variation. To study these factors without inheriting task and viewpoint priors from a particular robot-pretraining dataset, all experiments in Sections IV-C and IV-D initialize $\pi_{0.5}$ from pretrained VLM weights [54] without robot-action pretraining. For the representation comparison, we fix the fusion method to addition. For the fusion comparison, we fix the representation to pointmaps. These two comparisons use the robot-base frame for representations requiring a 3D coordinate frame, while coordinate-frame design is studied separately in Section IV-D.

TABLE II
CONTROLLED ABLATION OF 3D MODALITY AND FUSION METHOD.

3D Modality	Fusion Method	SR (%) (24 Tasks)	Δ vs. Ref.
RGB-only	—	25.6 ±1.3	−8.8
Depth	Addition	29.6 ±1.1	−4.8
Plücker	Addition	29.3 ±0.9	−5.1
Depth & Plücker	Addition	31.7 ±0.3	−2.7
Point Cloud	Addition	31.4 ±1.5	−3.0
Pointmap	Token Concat.	30.3 ±0.6	−4.1
Pointmap	Channel Concat.	31.6 ±1.4	−2.8
Pointmap	Addition	**34.4** ±1.3	—

1) How should 3D geometry be represented?: We first compare depth, camera, point clouds, and pointmaps as geometric inputs to the VLA in Tab. II. Depth provides per-pixel surface distance, while Plücker rays [41] encode camera geometry from intrinsics and extrinsics. Together, these signals contain the information required to recover 3D scene coordinates in the robot frame. Depth and Plücker rays individually improve over RGB alone (29.6 and 29.3 vs. 25.6), while combining them further increases success to 31.7. Explicitly converting the same information into robot-frame pointmaps further improves success to 34.4, suggesting a benefit from providing robot-aligned 3D coordinates rather than leaving their geometric relationship implicit.

We also compare against a point-cloud representation. We encode robot-frame points with a pretrained point-cloud encoder [56], align the resulting features with RGB patches following [9], and add them to the corresponding RGB tokens. This variant reaches 31.4, below the pointmap representation. Despite both providing robot-frame 3D geometry, the pointmap retains dense pixel-wise correspondence with RGB, which may contribute to its higher performance. Because the two variants also differ in their geometric encoders, this comparison does not isolate the effect of density alone. Overall, these results motivate our use of dense, precomputed robot-centric pointmaps.

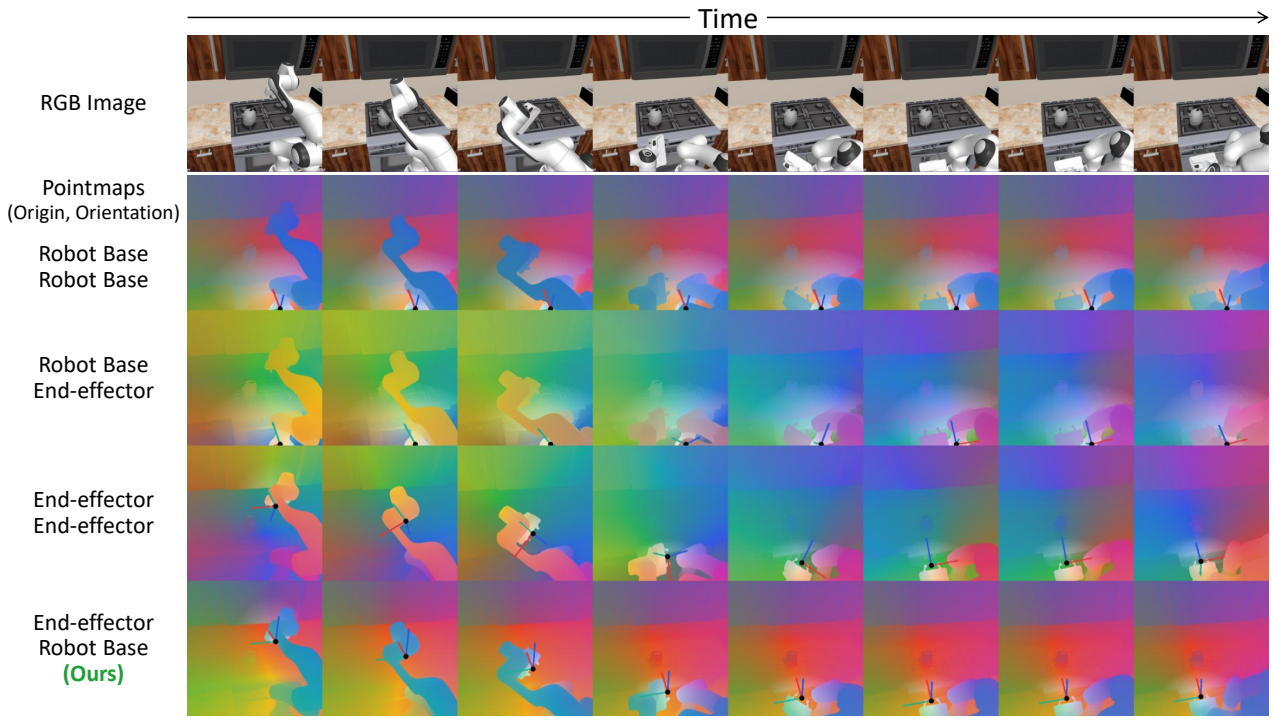


Fig. 4. **Qualitative comparison of pointmap coordinate frames along a RoboCasa rollout.** Each row specifies the origin followed by the axis orientation. An end-effector origin expresses objects relative to the gripper, providing a common reference for similar interactions across workspace locations. End-effector-aligned axes introduce wrist-induced coordinate changes (middle two pointmap rows), whereas our design preserves gripper-relative positions without these changes by retaining robot-base-aligned axes (bottom row).

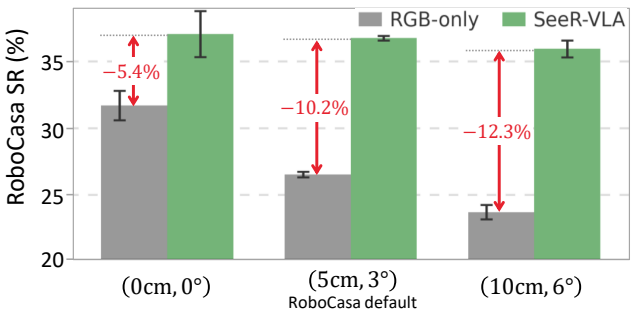


Fig. 5. **Increasing gains with training-time camera variation.** External-camera poses are randomized per demonstration with independent zero-mean Gaussian noise using per-axis translation/rotation standard deviations of $(0\,\mathrm{cm}, 0°)$, $(5\,\mathrm{cm}, 3°)$, or $(10\,\mathrm{cm}, 6°)$, while the wrist camera remains unchanged. As variation increases, SeeR-VLA maintains success while RGB-only performance declines, widening the gap.

2) How should pointmaps be fused with RGB?: We next compare channel-wise concatenation, token-wise concatenation, and addition for fusing pointmaps with RGB in Tab. II. Among the three strategies, addition performs best, reaching 34.4 compared with 30.3 for token-wise concatenation and 31.6 for channel-wise concatenation. Unlike channel-wise concatenation, addition allows RGB and pointmaps to be encoded separately while preserving one-to-one spatial correspondence between their visual tokens. It also avoids increasing the visual token sequence length, as in token-wise concatenation. These results suggest that separately encoding the two modalities and combining spatially aligned features is an effective way to incorporate pointmaps into the VLA.

3) How does training-time viewpoint variation affect performance?: We evaluate whether robot-centric pointmaps provide larger gains over RGB-only input as camera viewpoints in the training data become more diverse. For each training demonstration, we perturb the external-camera poses with independent zero-mean Gaussian noise, using per-axis translation/rotation standard deviations of $(0\,\mathrm{cm}, 0°)$, $(5\,\mathrm{cm}, 3°)$, or $(10\,\mathrm{cm}, 6°)$. The wrist camera and other training settings remain unchanged. As shown in Fig. 5, the success-rate gap between SeeR-VLA and RGB-only increases from 5.4 to 10.2 and 12.3 percentage points as viewpoint variation increases. RGB-only performance declines, whereas SeeR-VLA remains comparatively stable. These results suggest that providing 3D geometry in robot coordinates is particularly useful when learning from demonstrations collected across diverse camera viewpoints, making it well suited to training on large-scale robot datasets.

D. Which Coordinate Frame Should Pointmaps Use?

A pointmap coordinate frame is determined by its origin and axis orientation. We vary each independently between the robot base and end effector while fixing the pointmap representation and additive RGB fusion.

1) Qualitative Analysis: Fig. 4 illustrates how the origin and orientation affect pointmaps along a rollout. The first and last pointmap rows isolate the effect of changing the origin while retaining robot-base-aligned axes. With a robot-base origin, point coordinates describe absolute workspace locations, so the same gripper–object configuration can be represented differently when it occurs at different locations.

TABLE III

CONTROLLED ABLATION OF THE POINTMAP'S COORDINATE FRAMES.

Coordinate Frame		SR (%) (24 Tasks)	Δ vs. Ours
Origin	Orientation		
Robot Base	Robot Base	34.4 ±1.3	−2.4
Robot Base	End-effector	33.4 ±1.4	−3.4
End-effector	End-effector	<u>34.9</u> ±1.1	−1.9
End-effector	Robot Base	**36.8** ±0.2	—

Centering the pointmap at the end effector instead keeps the gripper at the origin and represents surrounding points by their displacement from it. Thus, similar interactions share similar geometric coordinates even across different workspace locations.

The middle two rows illustrate the effect of rotating the axes with the end effector. As the wrist rotates, end-effector-aligned axes rotate as well, causing coordinates to change even for static scene points. Moreover, the meaning of each coordinate axis changes with wrist orientation; for example, a positive z coordinate no longer consistently denotes the same robot-frame direction. Using robot-base-aligned axes avoids these wrist-induced changes while retaining the gripper-relative positions provided by end-effector centering. Our coordinate frame therefore combines a moving origin for interaction-relative geometry with fixed axes for consistent spatial directions.

2) Quantitative Comparison: Tab. III confirms this qualitative intuition. Moving the origin to the end effector improves success under both orientations, from 34.4% to 36.8% with robot-base axes and from 33.4% to 34.9% with end-effector axes. In contrast, aligning the axes with the end effector reduces success under both origins. The combination of an end-effector origin and robot-base axes performs best at 36.8%, supporting the use of gripper-relative coordinates with stable robot-base-aligned directions.

E. Analysis

Having examined the pointmap design, we now investigate whether the policy exploits object and robot coordinates in the pointmap to predict actions. We first examine whether SeeR-VLA uses pointmap geometry and which regions influence its actions. For observations from evaluation episodes, we compute gradients of the predicted end-effector motion a over the next 25 steps with respect to pointmap coordinates and RGB values. The resulting magnitudes, $|\partial a/\partial P(u,v)|$ and $|\partial a/\partial I(u,v)|$, indicate where small input changes most affect the predicted motion. As shown in Fig. 6 (a), pointmap sensitivity concentrates on the target object outlined in green, whereas RGB sensitivity is more broadly distributed. This localization suggests that SeeR-VLA uses the target's 3D coordinates for action prediction.

We next test whether removing this geometry impairs task execution. We replace robot-frame pointmap coordinates covering the target object and robot with a mask token while leaving RGB unchanged, and evaluate on eight RoboCasa pick-and-place tasks. This targeted masking reduces success from 52.9% to 42.1% (Fig. 6(b)). To determine whether this drop reflects the removal of task-relevant geometry rather than a generic masking effect, we evaluate two additional masking strategies. Shifted masking moves the same mask to a random, task-irrelevant location, while random masking removes the same number of scattered pixels. These strategies retain success rates of 51.5% and 52.5%, respectively, despite masking as many pixels as the target-object and robot mask. Together, the gradient maps and masking results show that SeeR-VLA relies on target-object and robot coordinates for action prediction.

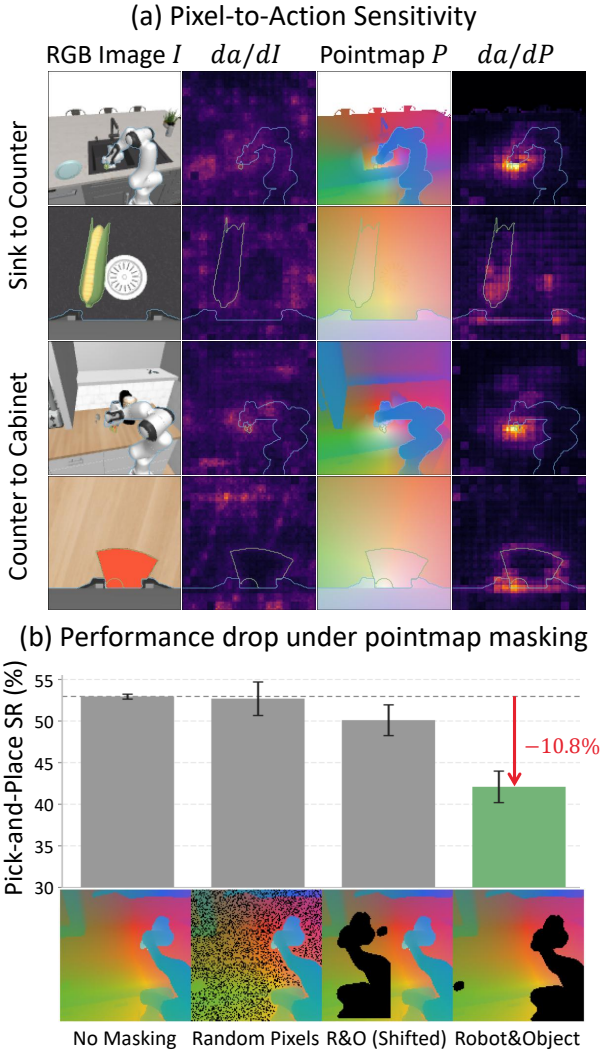


Fig. 6. **Policy reliance on task-relevant pointmap geometry.** Masking target-object and robot regions in pointmaps lowers success on eight pick-and-place tasks, unlike equal-sized shifted or random masks. Action gradients with respect to pointmap coordinates, $|\partial a/\partial P(u,v)|$, highlight target pixels where small coordinate changes most affect predicted actions.

V. CONCLUSION

We study how 3D geometry should be represented and integrated into VLAs trained on robot data collected from diverse camera viewpoints. Our results show that explicitly expressing geometry as dense, robot-centric pointmaps is more effective than leaving geometric relationships implicit or processing geometry as separate point clouds. Controlled ablations demonstrate the effectiveness of spatially aligned

fusion of separately encoded RGB and pointmap features, the benefits of end-effector-centered coordinates with robot-base-aligned axes, and increasing pointmap gains as training viewpoints diversify. This design improves pretrained VLAs in both simulation and real-world experiments, with analysis of the final policy demonstrating reliance on task-relevant pointmap geometry. While our method currently requires metric depth and camera calibration, future work could reduce these requirements by leveraging monocular geometry models [14]–[16] to estimate metric depth and intrinsics alongside camera-to-robot pose estimation. Overall, our results highlight that both the representation and coordinate frame of 3D observations matter, and that robot-centric pointmaps provide a simple way to align visual geometry with the space in which the robot acts.

REFERENCES

[1] B. Zitkovich *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *CoRL*, 2023. 1

[2] J. Bjorck *et al.*, "GR00T N1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025. 1

[3] K. Black, N. Brown *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," in *CoRL*, 2025. 1, 3, 4, 5

[4] J. Zhang *et al.*, "4D-VLA: Spatiotemporal vision-language-action pretraining with cross-scene calibration," in *NeurIPS*, 2025. 1, 2

[5] V. Bhat *et al.*, "3D CAVLA: Leveraging depth and 3D context to generalize vision language action models for unseen tasks," *arXiv preprint arXiv:2505.05800*, 2025. 1, 2

[6] A. Goyal *et al.*, "RVT-2: Learning precise manipulation from few demonstrations," in *RSS*, 2024. 1, 2

[7] C. Li *et al.*, "PointVLA: Injecting the 3D world into vision-language-action models," *RA-L*, 2026. 1, 2, 4, 5

[8] L. Sun *et al.*, "GeoVLA: Empowering 3D representations in vision-language-action models," *arXiv preprint arXiv:2508.09071*, 2025. 1, 2, 4, 5

[9] X. Fan *et al.*, "Any3D-VLA: Enhancing VLA robustness via diverse point clouds," *ICML*, 2026. 1, 2, 5

[10] D. Hwang *et al.*, "3D HAMSTER: Bridging planning and control in hierarchical vision language action models through 3D trajectory guidance," *IROS*, 2026. 1, 2

[11] T.-W. Ke *et al.*, "3D diffuser actor: Policy diffusion with 3D scene representations," in *CoRL*, 2024. 1, 2

[12] N. Gkanatsios *et al.*, "3D FlowMatch Actor: Unified 3D policy for single-and dual-arm manipulation," *arXiv preprint arXiv:2508.11002*, 2025. 1, 2

[13] S. Wang *et al.*, "DUSt3R: Geometric 3D vision made easy," in *CVPR*, 2024. 1, 2

[14] R. Wang *et al.*, "MoGe: Unlocking accurate monocular geometry estimation for open-domain images with optimal training supervision," in *CVPR*, 2025. 1, 2, 8

[15] ——, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," in *NeurIPS*, 2025. 1, 2, 8

[16] J. Wang *et al.*, "VGGT: Visual geometry grounded transformer," in *CVPR*, 2025. 1, 2, 8

[17] Q. Wang *et al.*, "Continuous 3D perception model with persistent state," in *CVPR*, 2025. 1, 2

[18] Y. Wang *et al.*, "π^3: Permutation-equivariant visual geometry learning," in *ICLR*, 2026. 1, 2

[19] X. Jia *et al.*, "PointMapPolicy: Structured point cloud processing for multi-modal imitation learning," in *NeurIPS*, 2025. 1, 2

[20] X. Yang *et al.*, "ReMAP-DP: Reprojected multi-view aligned PointMaps for diffusion policy," *arXiv preprint arXiv:2603.14977*, 2026. 1, 2

[21] G. Yan *et al.*, "Flex-π: A multi-stream world-action model with compute flexibility," *arXiv preprint arXiv:2608.10860*, 2026. 1, 2

[22] A. O'Neill *et al.*, "Open X-Embodiment: Robotic learning datasets and RT-X models," in *ICRA*, 2024. 1

[23] Q. Bu *et al.*, "Agibot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems," in *IROS*, 2025. 1

[24] A. Khazatsky *et al.*, "DROID: A large-scale in-the-wild robot manipulation dataset," in *RSS*, 2024. 1, 4

[25] Y. Feng *et al.*, "Demystifying action space design for robotic manipulation policies," *arXiv preprint arXiv:2602.23408*, 2026. 1

[26] M. Liu *et al.*, "Frame mining: a free lunch for learning robotic manipulation from 3D point clouds," in *CoRL*, 2022. 1, 2

[27] W. Yuan *et al.*, "M2T2: Multi-task masked transformer for object-centric pick and place," *CoRL*, 2023. 2

[28] Y. Ze *et al.*, "3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations," in *RSS*, 2024. 2

[29] S. Chen *et al.*, "PointACT: Vision-language-action models with multi-scale point-action interaction," in *RSS*, 2026. 2

[30] M. Shridhar *et al.*, "Perceiver-Actor: A multi-task transformer for robotic manipulation," in *CoRL*, 2022. 2

[31] T. Gervet *et al.*, "Act3D: 3D feature field transformers for multi-task robotic manipulation," in *CoRL*, 2023. 2

[32] F. Liu *et al.*, "RoboUniView: Visual-language model with unified view representation for robotic manipulation," *arXiv preprint arXiv:2406.18977*, 2024. 2

[33] V. Leroy *et al.*, "Grounding image matching in 3D with MASt3R," in *ECCV*, 2024. 2

[34] J. Yang *et al.*, "Fast3R: Towards 3D reconstruction of 1000+ images in one forward pass," in *CVPR*, 2025. 2

[35] R. Yang *et al.*, "FP3: A 3D foundation policy for robotic manipulation," *arXiv preprint arXiv:2503.08950*, 2025. 2, 4, 5

[36] L. Ke *et al.*, "Grasping with chopsticks: Combating covariate shift in model-free imitation learning for fine manipulation," in *ICRA*, 2021. 2

[37] K. Rana *et al.*, "Learning from 10 demos: Generalisable and sample-efficient policy learning with oriented affordance frames," in *CoRL*, 2025. 2

[38] Z. Zhang *et al.*, "Canonical policy: Learning canonical 3D representation for equivariant policy," *arXiv preprint arXiv:2505.18474*, 2025. 2

[39] A. Wilcox *et al.*, "Adapt3R: Adaptive 3D scene representation for domain transfer in imitation learning," in *CoRL*, 2025. 2

[40] H. Lin *et al.*, "Universal pose pretraining for generalizable vision-language-action policies," in *RSS*, 2026. 2

[41] T. Jiang *et al.*, "Do you know where your camera is? View-invariant policy learning with camera conditioning," in *ICRA*, 2026. 2, 4, 5

[42] T. Zhang *et al.*, "Grounding actions in camera space: Observation-centric vision-language-action policy," in *AAAI*, 2026. 2, 4, 5

[43] D. Qu *et al.*, "SpatialVLA: Exploring spatial representations for visual-language-action model," in *RSS*, 2025. 2

[44] T. Yuan *et al.*, "DepthVLA: Enhancing vision-language-action models with depth-aware spatial reasoning," *arXiv preprint arXiv:2510.13375*, 2025. 2

[45] P. Li *et al.*, "BridgeVLA: Input-output alignment for efficient 3D manipulation learning with vision-language models," in *NeurIPS*, 2025. 2

[46] I. Singh *et al.*, "OG-VLA: Orthographic image generation for 3D-aware vision-language action model," *arXiv preprint arXiv:2506.01196*, 2025. 2

[47] H. Zhen *et al.*, "3D-VLA: 3D vision-language-action generative world model," in *ICML*, 2024. 2

[48] T. Lin *et al.*, "Evo-0: Vision-language-action model with implicit spatial understanding," *arXiv preprint arXiv:2507.00416*, 2025. 2

[49] F. Li *et al.*, "Spatial forcing: Implicit spatial representation alignment for vision-language-action model," in *ICLR*, 2026. 2

[50] X. Liu *et al.*, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *ICLR*, 2023. 3

[51] S. Nasiriany *et al.*, "RoboCasa: Large-scale simulation of everyday tasks for generalist robots," in *RSS*, 2024. 4, 5

[52] X. Zhai *et al.*, "Sigmoid loss for language image pre-training," in *ICCV*, 2023. 4

[53] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *ICLR*, 2019. 4

[54] L. Beyer *et al.*, "PaliGemma: A versatile 3B VLM for transfer," *arXiv preprint arXiv:2407.07726*, 2024. 4, 5

[55] B. Wen *et al.*, "Fast-FoundationStereo: Real-time zero-shot stereo matching," *CVPR*, 2026. 4

[56] X. Wu *et al.*, "Sonata: Self-supervised learning of reliable point representations," in *CVPR*, 2025. 5